5 Conditional Validation

Sometimes it will make sense to validate an object only when a given predicate is satisfied. You can do that by using the `:if` and `:unless` options, which can take a symbol, a string, a `Proc` or an `Array`. You may use the `:if` option when you want to specify when the validation **should** happen. If you want to specify when the validation **should not** happen, then you may use the `:unless` option.

5.1 Using a Symbol with `:if` and `:unless`

You can associate the `:if` and `:unless` options with a symbol corresponding to the name of a method that will get called right before validation happens. This is the most commonly used option.

```
class Order < ActiveRecord::Base
  validates :card_number, presence: true, if: :paid_with_card?

  def paid_with_card?
    payment_type == "card"
  end
end
```

5.2 Using a String with `:if` and `:unless`

You can also use a string that will be evaluated using `eval` and needs to contain valid Ruby code. You should use this option only when the string represents a really short condition.

```
class Person < ActiveRecord::Base
  validates :surname, presence: true, if: "name.nil?"
end
```

5.3 Using a Proc with `:if` and `:unless`

Finally, it's possible to associate `:if` and `:unless` with a `Proc` object which will be called. Using a `Proc` object gives you the ability to write an inline condition instead of a separate method. This option is best suited for one-liners.

```
class Account < ActiveRecord::Base
  validates :password, confirmation: true,
    unless: Proc.new { |a| a.password.blank? }
end
```

5.4 Grouping Conditional validations

Sometimes it is useful to have multiple validations use one condition, it can be easily achieved using `with_options`.

```
class User < ActiveRecord::Base
  with_options if: :is_admin? do |admin|
    admin.validates :password, length: { minimum: 10 }
    admin.validates :email, presence: true
  end
end
```

All validations inside of `with_options` block will have automatically passed the condition `if: :is_admin?`

5.5 Combining Validation Conditions

On the other hand, when multiple conditions define whether or not a validation should happen, an `Array` can be used. Moreover, you can apply both `:if` and `:unless` to the same validation.

```
class Computer < ActiveRecord::Base
  validates :mouse, presence: true,
```